| | |
|---|---|
| **Date** | 16 July 2026 |
| **Project Name** | Smart Lender AI: An Intelligent Machine Learning System for Loan Approval Prediction |
| **Student Name** | Jinkala Sunitha |
| **Maximum Marks** | 3 Marks |

# REQUIREMENT ANALYSIS: TECHNOLOGY STACK SELECTION

To ensure that the **Smart Lender AI** platform meets its targeted performance thresholds—specifically achieving sub-second backend classification latency while rendering interactive dashboards—the technology stack has been selected for optimal modularity, integration, and development speed.

## Stack Allocation & Rationale

| LAYER | SELECTED TECHNOLOGY | RATIONALE & PROJECT ALIGNMENT |
|---|---|---|
| **Frontend & UI/UX** | **HTML5, CSS3, JavaScript (Bootstrap 5)** | Enables a fully responsive, clean dashboard UI to show model outputs. Eliminates heavy rendering framework overheads (like React/Angular), allowing immediate server-side integration. |
| **Backend API** | **Flask (Python 3.10+)** | Provides an extremely lightweight web framework ideal for serving ML models. Facilitates low routing latency (<10ms processing overhead) to handle incoming transient validation requests. |
| **Data Processing & Math** | **Pandas, NumPy, Scikit-learn** | Offers robust and standard mathematical processing arrays. Used to handle identical vector preprocessing (mean imputation, standard scaling) in testing that matches training parameters. |
| **Machine Learning Engines** | **XGBoost & Random Forest (Pickled)** | Selected for high classification F1-scores and optimized multi-threaded CPU matrix operations, supporting fast, consistent, and reliable local evaluations. |
| **Serialization & Tooling** | **Joblib / Pickle, Git/ GitHub, Virtualenv** | Enables binary compression of trained model pipelines to disk, guaranteeing portable execution. Version control and isolated environments ensure clean workspace portability. |

## Architectural Decisions

### Python-Centric Execution:

Using an end-to-end Python environment (from *Jupyter* for model prototyping down to *Flask* for active API deployment) completely avoids costly cross-language cross-compilations. The exact scikit-learn structures and scaling parameters exported during model fitting are loaded directly into active API memory, minimizing potential vector alignment bugs during execution.